

HAI802I Algorithmique avancée — DM

Ivan Lejeune

4 mai 2026

Table des matières

1	Complexité	2
1.1	Problèmes NP-complets	2
1.2	Points extrêmes	4
1.3	<i>AP</i> -réduction	6

1 Complexité

1.1 Problèmes NP-complets

Exercice 1 Problème du Feedback Vertex Set.

Le problème du FEEDBACK VERTEX SET défini ci-dessous est un problème fondamental en base de données. Dans les systèmes d'exploitation, ce problème joue un rôle important dans l'étude de la récupération de données après des crashes ou des blocages. Dans le graphe des processus d'un système d'exploitation, chaque cycle dirigé correspond à une situation d'impasse. Afin de résoudre toutes les impasses, certains processus bloqués doivent être abandonnés. Un FEEDBACK VERTEX SET dans le graphe correspond à un nombre minimum de processus qu'il faut abandonner.

Problème 1 – FEEDBACK VERTEX SET

Input: Soit $G = (V, E)$ un graphe orienté et $k \in \mathbb{N}$

Question: Existe-t-il un ensemble $S \subset V$ de k sommets tel que $G \setminus S$ est un graphe sans circuit (tous les arcs sont orientés dans le même sens)?

On considère les figures ci-dessous :

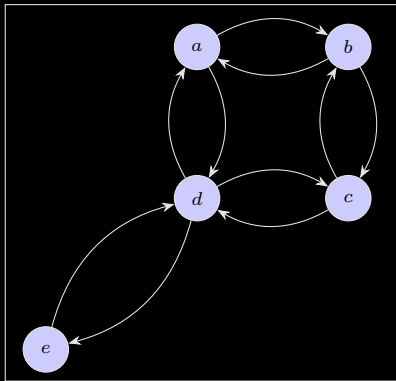


FIGURE 1 – Exemple pour le FEEDBACK VERTEX SET

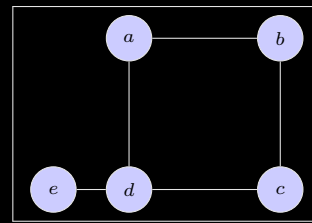


FIGURE 2 – Instance initiale pour le VERTEX COVER

1. Donner l'ensemble S pour le graphe orienté donné par la figure 1.
2. Proposer une réduction à partir de VERTEX COVER et effectuer la preuve de NP-complétude.
3. Appliquer votre réduction sur le graphe donné par la figure 2.

Solution.

1. On commence par essayer de trouver un ensemble S de taille $k = 1$. Si on retire e ou d alors le cycle $a - b$ subsiste. Sinon c'est le cycle $e - d$ qui subsiste. Il n'y a donc pas de solution pour $k = 1$. On cherche maintenant une solution pour $k = 2$. Il faut casser chaque cycle dans le graphe donc il faut pour chaque paire de sommets adjacents dans le graphe, retirer un des deux sommets. Si on choisit $S = \{b, d\}$ il n'y a plus de cycles dans le graphe $G \setminus S = \{a, c, e\}$. Alors $S = \{b, d\}$ est bien solution minimale du problème avec $k = 2$.
2. Commençons par montrer que le problème de FEEDBACK VERTEX SET appartient à la classe NP. Soit S une solution au problème de FEEDBACK VERTEX SET. On commence par vérifier que S est bien de taille k (se fait en temps linéaire). Ensuite, pour vérifier la validité d'une telle solution, on retire S du graphe G (se fait en temps linéaire) puis on effectue un BFS sur chaque sommet de G (polynomial en le nombre de sommets) pour vérifier qu'on ne retombe pas sur le sommet de départ. Ainsi le problème de FEEDBACK VERTEX SET est bien NP.

Montrons maintenant qu'on peut réduire le problème de VERTEX COVER à un problème de FEEDBACK VERTEX SET. Soit $G = (V, E)$ un graphe non orienté. On construit le graphe $G' = (V', E')$ comme suit : pour chaque sommet $v \in V$ de G , on rajoute $v' = v \in V'$. Pour chaque arête $u, v \in E$, on rajoute deux arcs orientés u, v, v, u dans E' . Alors $G' = (V', E')$ correspond au graphe G où chaque arête est remplacée par deux arcs de sens opposé. Cette construction peut se faire en temps polynomial.

On commence par montrer le sens direct. Soit S une solution au VERTEX COVER dans G . Alors, toute arête $u, v \in E$ a bien $u \in S$ ou $v \in S$. Alors, tous les 2-cycles dans G' sont cassés (sinon l'arête correspondante ne serait pas couverte par S). Alors tous les sommets de G' sont isolés et G' n'a donc aucun cycle. On a trouvé une solution à FEEDBACK VERTEX SET dans G' à partir d'une solution de VERTEX COVER dans G .

Maintenant le sens indirect. Soit S une solution de FEEDBACK VERTEX SET dans G' . Alors, toute arête $u, v \in E'$ a bien $u \in S$ ou $v \in S$ (sinon $u - v - u$ formerait un cycle). Alors, toutes les arêtes dans E sont bien couvertes par S dans G (sinon on aurait un 2-cycle induit dans G'). On a trouvé une solution à VERTEX COVER dans G à partir d'une solution de FEEDBACK VERTEX SET dans G' .

Alors, le problème de FEEDBACK VERTEX SET est NP-complet.

3. Appliquons la réduction au graphe 3.

On commence par créer le graphe G' comme décrit dans la réduction :

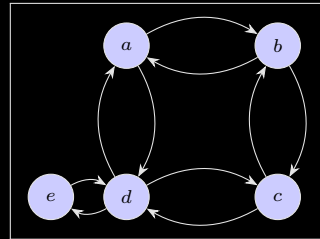


FIGURE 3 – Graphe G' obtenu par réduction

Une solution au problème de VERTEX COVER dans le graphe de départ est $S = \{d, b\}$. Si on retire S au graphe G' on obtient :

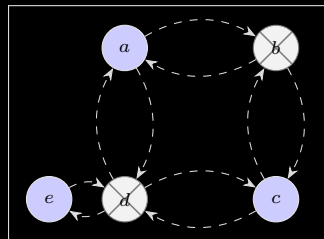


FIGURE 4 – Graphe G' obtenu par réduction

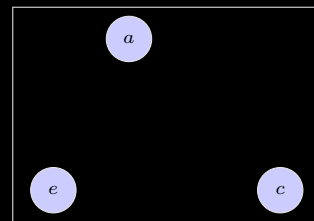


FIGURE 5 – Graphe G' simplifié

Exercice 2 Réduction polynomiale. On considère le problème de MINIMAL SQUARE SUM défini de la manière suivante :

Problème 2 – MINIMAL SQUARE SUM

Input: Un ensemble fini A , une fonction $s : A \rightarrow \mathbb{N}$ et deux entiers k et j .

Question: Peut-on partitionner les éléments de A en k sous-ensembles $A_1, A_2, \dots, A_k$ tels que $\sum_{i=1}^k (\sum_{a \in A_i} s(a))^2 \leq j$?

Montrer que le problème MINIMAL SQUARE SUM est NP-complet. La preuve se fait à partir du problème de PARTITION PROBLEM qui est lui aussi NP-complet et défini de la manière

suivante :

Problème 3 – PARTITION PROBLEM

Input: Un ensemble fini A et une fonction $s : A \rightarrow \mathbb{N}$.

Question: Existe-t-il un sous-ensemble $A' \subseteq A$ tel que $\sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a)$?

Solution. Soit A un ensemble fini et $s : A \rightarrow \mathbb{N}$ une fonction qui encode le poids des éléments de A .

On construit alors une instance du MINIMAL SQUARE SUM de la manière suivante :

- On garde le même ensemble d'éléments A ,
- on garde la même fonction de poids s ,
- on pose $k = 2$,
- on pose $j = \frac{S^2}{2}$ où S représente le poids total de A .

Alors, pour résoudre le problème de MINIMAL SQUARE SUM, il faut trouver A' tel que :

$$\begin{aligned} f(x) &= \sum_{i=1}^k \left(\sum_{a \in A_i} s(a) \right)^2 \leq j \\ &= \sum_{i=1}^2 \left(\sum_{a \in A_i} s(a) \right)^2 \leq \frac{S^2}{2} \\ &= \left(\sum_{a \in A'} s(a) \right)^2 + \left(\sum_{a \in A \setminus A'} s(a) \right)^2 \leq \frac{S^2}{2} \end{aligned}$$

Soit x le poids total de l'ensemble A' . On a alors

$$\begin{aligned} f(x) &= \left(\sum_{a \in A'} s(a) \right)^2 + \left(\sum_{a \in A \setminus A'} s(a) \right)^2 \\ &= x^2 + (S - x)^2 \\ &= x^2 + S^2 + x^2 - 2Sx \\ &= 2x^2 - 2Sx + S^2 \end{aligned}$$

Cette fonction trouve son minimum en $x = \frac{S}{2}$ (qu'on peut trouver par exemple en cherchant $f'(x) = 0$). Pour minimiser le poids total des carrés il faut donc trouver une partition des poids en exactement $f\left(\frac{S}{2}\right) = \frac{S^2}{2}$.

Si une partition parfaite existe alors on a $f(x) = \frac{S^2}{2}$. Sinon, on a $f(x) > \frac{S^2}{2}$ (car on est en dehors du minimum). Plus précisément, comme on a posé $j = \frac{S^2}{2}$, on vient de montrer que

$$f \leq j \iff \text{Une partition en deux ensemble de poids égal existe}$$

On a donc construit une réduction polynomiale du problème de PARTITION PROBLEM au problème de MINIMAL SQUARE SUM.

Comme PARTITION PROBLEM est NP-complet et MINIMAL SQUARE SUM est NP, MINIMAL SQUARE SUM est NP-complet.

1.2 Points extrêmes

Exercice 3 Vertex Cover dans les graphes bipartis et points extrêmes.

1. Donner le programme linéaire en nombres entiers pour le problème du VERTEX COVER.

2. On veut montrer que dans le cas où G est un graphe biparti $G = ((A \sqcup B), E)$ alors l'ensemble des points extrêmes admet des valeurs entières.

(a) Pour cela, nous allons procéder à une preuve par contraposée. Supposer que x est un point extrême défini de la manière suivante : $x = (x_1, x_2, \dots, x_n)$ admet au moins une coordonnée non entière. Supposons qu'il existe une coordonnée $x_v = \frac{1}{2}$ dans x . Soit $t = (t_1, \dots, t_n)$ et $w = (w_1, \dots, w_n)$ deux nouveaux points définis de la manière suivante :

$$t_i = \begin{cases} x_i & \text{si } x_i \text{ entier} \\ 0 & \text{si } x_i = \frac{1}{2} \text{ et } i \in A \\ 1 & \text{si } x_i = \frac{1}{2} \text{ et } i \in B \end{cases} \quad w_i = \begin{cases} x_i & \text{si } x_i \text{ entier} \\ 0 & \text{si } x_i = \frac{1}{2} \text{ et } i \in B \\ 1 & \text{si } x_i = \frac{1}{2} \text{ et } i \in A \end{cases}$$

Vérifier que

$$x = \frac{1}{2}t + \frac{1}{2}w.$$

(b) Montrer que t et w sont deux points extrêmes.

3. Conclure sur l'existence d'un algorithme polynomial pour résoudre le VERTEX COVER dans les graphes bipartis.

Solution.

1. On rappelle que le programme linéaire associé au problème de VERTEX COVER s'énonce comme suit :

$$ILP = \begin{cases} \min(z) = \sum_{i=1}^n x_i \\ x_i + x_j \geq 1, & \forall (x_i, x_j) \in E \\ x_i \in \{0, 1\}, & \forall i \in \{1, \dots, n\} \end{cases}$$

où n correspond au nombre de sommets de G .

2. On montre dans la suite que dans le cas d'un graphi biparti, les points extrêmes sont à valeurs entières :

(a) On procède par disjonction de cas :

• Si x_i est entier alors on a

$$\begin{aligned} x_i &= \frac{1}{2}t_i + \frac{1}{2}w_i \\ &= \frac{1}{2}x_i + \frac{1}{2}x_i \\ &= x_i \end{aligned}$$

• Si x_i est non entier et $i \in A$ alors on a

$$\begin{aligned} x_i &= \frac{1}{2}t_i + \frac{1}{2}w_i \\ &= \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 1 \\ &= \frac{1}{2} \\ &= x_i \end{aligned}$$

- Si x_i est non entier et $i \in B$ alors on a

$$\begin{aligned} x_i &= \frac{1}{2}t_i + \frac{1}{2}w_i \\ &= \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 0 \\ &= \frac{1}{2} \\ &= x_i \end{aligned}$$

Alors, pour tout i on a $x_i = \frac{1}{2}t_i + \frac{1}{2}w_i$ donc

$$x = \frac{1}{2}t + \frac{1}{2}w$$

(b) Montrons que t et w satisfont les contraintes du programme linéaire associé au problème de VERTEX COVER. Soit $(i, j) \in E$ une arête de G . Alors, on a les cas suivants :

- Si x_i et x_j sont entiers alors $t_i = w_i = x_i$ et $t_j = w_j = x_j$ donc $t_i + t_j = w_i + w_j = x_i + x_j \geq 1$.
- S'ils sont tous deux égaux à $\frac{1}{2}$ car G est un graphe biparti (donc i et j ne peuvent pas être dans le même ensemble A ou B). Alors, on a $t_i + t_j = 0 + 1 = 1$ et $w_i + w_j = 1 + 0 = 1$.
- Si au plus un des deux sommets i ou j est entier alors on a $t_i + t_j \geq 1$ et $w_i + w_j \geq 1$ (car au moins un des deux sommets est égal à 1 dans t et dans w).

Alors, t et w satisfont les contraintes du programme linéaire associé au problème de VERTEX COVER.

Cela contredit l'hypothèse que x est un point extrême car il est alors combinaison convexe de t et w qui sont deux points différents de x . Comme il existe au moins un sommet v tel que $x_v = \frac{1}{2}$, on a $t_v \neq w_v$, donc $t \neq w$. Donc, x n'est pas un point extrême. Donc, tous les points extrêmes du programme linéaire associé au problème de VERTEX COVER dans les graphes bipartis sont à valeurs entières.

3. Comme tous les points extrêmes du programme linéaire associé au problème de VERTEX COVER dans les graphes bipartis sont à valeurs entières, alors on peut résoudre le problème de VERTEX COVER dans les graphes bipartis en résolvant son programme linéaire associé (en utilisant un algorithme polynomial comme l'ellipsoïde).

Il existe donc un algorithme polynomial pour résoudre le problème de VERTEX COVER dans les graphes bipartis.

1.3 AP-réduction

Exercice 7 Sur le produit matriciel.

Appelons (p, q) -matrice une matrice à p lignes et q colonnes. Nous admettons que le produit d'une (p, q) -matrice par une (q, r) -matrice peut se faire à l'aide de pqr opérations.

1. Soit M_i une (p_i, p_{i+1}) -matrice pour $i = 1, \dots, k$. Nous effectuons le produit suivant :

$$(M1(M2(\dots(M_{k-1}M_k))))$$

et

$$((\dots(M1M2)M3)\dots M_k)$$

Evaluer le nombre d'opérations nécessaires pour chacun de ces produits. Que remarque-t-on ?

2. Montrer que le nombre $c(k)$ de parenthésages possibles d'un produit de k matrices vérifie,

si on pose $c(1) = 1$,

$$c(k) = \sum_{i=1}^{k-1} c(i)c(k-i)$$

Nous admettrons que $c(k)$ est de l'ordre de $\frac{4^{k-1}}{k\sqrt{\pi k}}$ (ceci peut être obtenu en résolvant la récurrence puis à l'aide de la formule de Stirling). Une description exhaustive de tous les parenthésages d'un produit de k matrices, dans le but de déterminer celui qui permet d'effectuer le produit avec un nombre minimum d'opérations, nécessite donc un temps de calcul déraisonnablement exponentiel. La programmation dynamique permet de résoudre ce problème (d'un parenthésage optimum) en temps polynomial.

3. En considérant les blocs $M_{i,j} = M_i M_{i+1} \dots M_j$ pour $1 \leq i \leq j \leq k$, et en désignant par $m_{i,j}$ le coût minimum du calcul du produit $M_{i,j}$, établir la récurrence permettant d'appliquer la programmation dynamique pour obtenir $m_{1,k}$ et le parenthésage correspondant.
4. Appliquer ce principe pour $k = 5$ et $p_1 = 1, p_2 = 2, p_3 = 5, p_4 = 10, p_5 = 4$ et $p_x = 100$ (nous n'omettrons pas de construire la table correspondante et d'indiquer précisément le parenthésage optimal).

Solution.

1. Le nombre d'opérations nécessaires pour le premier produit est :

$$\begin{aligned}
 & (k-1)k(k+1) + (k-2)(k-1)(k+1) \\
 & \quad + (k-3)(k-2)(k+1) \\
 & \quad + \dots \\
 & \quad + 2 \cdot 3 \cdot (k+1) \\
 & \quad + 1 \cdot 2 \cdot (k+1) \\
 & = (k+1) \sum_{i=1}^{k-1} i(i+1) \\
 & = (k+1) \sum_{i=1}^{k-1} (i^2 + i) \\
 & = (k+1) \left(\frac{(k-1)k(2k-1)}{6} + \frac{(k-1)k}{2} \right) \\
 & = (k+1) \frac{(k-1)k(2k+2)}{6} \\
 & = \frac{(k-1)k(k+1)^2}{6} \\
 & = O(k^4)
 \end{aligned}$$

Le nombre d'opérations nécessaires pour le second produit est :

$$\begin{aligned}
& 1 \cdot 2 \cdot 3 + 1 \cdot 3 \cdot 4 \\
& \quad + \dots \\
& \quad + 1 \cdot (k-1) \cdot k \\
& \quad + 1 \cdot k \cdot (k+1) \\
& = \sum_{i=2}^k i(i+1) \\
& = \sum_{i=2}^k (i^2 + i) \\
& = \frac{k(k+1)(2k+1)}{6} + \frac{k(k+1)}{2} - 2 \\
& = O(k^3)
\end{aligned}$$

On remarque que l'ordre des produits a une influence significative sur le nombre d'opérations nécessaires pour effectuer le produit matriciel. Ici on note une différence d'ordre $O(k^4)$ contre $O(k^3)$.

2. On peut remarquer que choisir un parenthésage pour un produit de k matrices revient à choisir le dernier produit à effectuer (et puis de parenthéser les sous-produits), c'est à dire à choisir un i tel que $1 \leq i < k$ et à effectuer le produit de $M_1 M_2 \dots M_i$ puis le produit de $M_{i+1} M_{i+2} \dots M_k$. Comme il y a $c(i)$ façons de parenthéser le produit de i matrices et $c(k-i)$ façons de parenthéser le produit de $k-i$ matrices, il y a $c(i)c(k-i)$ façons de parenthéser le produit de k matrices en choisissant i comme dernier produit à effectuer. En sommant sur tous les i possibles, on obtient la récurrence suivante :

$$c(k) = \sum_{i=1}^{k-1} c(i)c(k-i)$$

3. En choisissant i comme dernier produit à effectuer pour le produit de $M_i M_{i+1} \dots M_j$, on doit effectuer le produit de $M_i M_{i+1} \dots M_k$ puis le produit de $M_{k+1} M_{k+2} \dots M_j$ puis le produit de ces deux produits. Le coût de ce parenthésage est donc $m_{i,k} + m_{k+1,j} + p_i p_{k+1} p_{j+1}$. En minimisant sur tous les i possibles, on obtient la récurrence suivante :

$$m_{i,j} = \min_{i \leq k < j} (m_{i,k} + m_{k+1,j} + p_i p_{k+1} p_{j+1})$$

avec $m_{i,i} = 0$ pour tout i .

4. On travaille donc avec des matrices de tailles :

$$\begin{aligned}
M_1 &: 1 \times 2, M_2 : 2 \times 5, \\
M_3 &: 5 \times 10, M_4 : 10 \times 4, \\
M_5 &: 4 \times 100
\end{aligned}$$

Cas de base : $m_{i,i} = 0$ pour $i = 1, \dots, 5$.

Cas de $m_{i,j}$ pour $j = i + 1$:

$$\begin{aligned}
m_{1,2} &= 1 \cdot 2 \cdot 5 = 10, \\
m_{2,3} &= 2 \cdot 5 \cdot 10 = 100, \\
m_{3,4} &= 5 \cdot 10 \cdot 4 = 200, \\
m_{4,5} &= 10 \cdot 4 \cdot 100 = 4000
\end{aligned}$$

Cas de $m_{i,j}$ pour $j = i + 2$:

$$\begin{aligned}
m_{1,3} &= \min(m_{1,1} + m_{2,3} + 1 \cdot 2 \cdot 10, m_{1,2} + m_{3,3} + 1 \cdot 5 \cdot 10) \\
&= \min(120, 60) = 60, \\
m_{2,4} &= \min(m_{2,2} + m_{3,4} + 2 \cdot 5 \cdot 4, m_{2,3} + m_{4,4} + 2 \cdot 10 \cdot 4) \\
&= \min(240, 180) = 180, \\
m_{3,5} &= \min(m_{3,3} + m_{4,5} + 5 \cdot 10 \cdot 100, m_{3,4} + m_{5,5} + 5 \cdot 4 \cdot 100) \\
&= \min(9000, 2200) = 2200
\end{aligned}$$

Cas de $m_{i,j}$ pour $j = i + 3$:

$$\begin{aligned}
m_{1,4} &= \min(m_{1,1} + m_{2,4} + 1 \cdot 2 \cdot 4, m_{1,2} + m_{3,4} + 1 \cdot 5 \cdot 4, m_{1,3} + m_{4,4} + 1 \cdot 10 \cdot 4) \\
&= \min(188, 230, 100) = 100, \\
m_{2,5} &= \min(m_{2,2} + m_{3,5} + 2 \cdot 5 \cdot 100, m_{2,3} + m_{4,5} + 2 \cdot 10 \cdot 100, m_{2,4} + m_{5,5} + 2 \cdot 4 \cdot 100) \\
&= \min(3200, 6100, 980) = 980
\end{aligned}$$

Cas de $m_{i,j}$ pour $j = i + 4$:

$$\begin{aligned}
m_{1,5} &= \min \begin{pmatrix} m_{1,1} + m_{2,5} + 1 \cdot 2 \cdot 100, \\ m_{1,2} + m_{3,5} + 1 \cdot 5 \cdot 100, \\ m_{1,3} + m_{4,5} + 1 \cdot 10 \cdot 100, \\ m_{1,4} + m_{5,5} + 1 \cdot 4 \cdot 100 \end{pmatrix} \\
&= \min(1180, 2710, 5060, 500) \\
&= 500
\end{aligned}$$

La table correspondante est donc :

$i \backslash j$	1	2	3	4	5
1	0	10	60	100	500
2		0	100	180	980
3			0	200	2200
4				0	4000
5					0

Pour obtenir le parenthésage optimal, il suffit de regarder les couts min à chaque étape. Pour minimiser le produit de $M_1 M_2 M_3 M_4 M_5$, on effectue d'abord $(M_1 \cdots M_4) M_5$ (cout min). Pour minimiser le produit de $M_1 M_2 M_3 M_4$, on effectue d'abord $(M_1 M_2 M_3) M_4$ (cout min). Pour minimiser le produit de $M_1 M_2 M_3$, on effectue d'abord $(M_1 M_2) M_3$ (cout min). Le parenthésage optimal est donc :

$$(((M_1 M_2) M_3) M_4) M_5$$